

Lezione — Modulo S1: Principi di Offensive Security e Enumerazione

Corso: Lab Sicurezza Informatica T **Materiale:** [Principi_delloffensive_security_20_febbraio.pdf](#)
· [Introduzione_alla_materia_20_febbraio.pdf](#) **Prerequisiti:** nessun modulo Security precedente; utile familiarità con reti TCP/IP e comandi Linux base (SysAdmin 0A–3C) **Nota esame:** contribuisce al **quiz teorico (40%)** — risposte errate penalizzano; se non sei sicuro, non rispondere.

Come leggere questa lezione

S1 è il modulo fondativo: introduce il linguaggio con cui parlano tutti gli altri. I ganci concreti sono tre — **nmap**, **nslookup/dnsrecon**, e **Google Dorking** — attorno ai quali si costruisce l'intera prospettiva offensiva e difensiva del corso. Leggi questa lezione la sera; la guida-lab ([/lab S1](#)) ti accompagna passo passo sulla VM.

La visione d'insieme: perché esiste l'offensive security

L'offensive security nasce da una constatazione scomoda: difendere bene richiede di capire come attacca il nemico. Un difensore che non ha mai “pensato da attaccante” tende a lasciare aperture ovvie proprio dove non guarda. La pratica di simulare attacchi reali — con permesso e obiettivi definiti — è il **penetration test**.

Il **penetration test (PT)** è un assessment autorizzato in cui un team simula le mosse di un attaccante reale su un sistema reale, con l'obiettivo di trovare vulnerabilità prima che le trovi qualcuno di ostile. “Autorizzato” è la parola chiave: le stesse tecniche, senza autorizzazione, sono reati. Il PT si distingue dalla **Vulnerability Assessment (VA)** perché non si limita a elencare le vulnerabilità note: le sfrutta davvero, verifica le catene di attacco, e produce un report che mostra l'impatto reale. Il **Red Team** è un passo ulteriore: un'operazione prolungata (settimane o mesi) che simula un attore ostile sofisticato, incluse tecniche di social engineering e movimenti laterali non rilevabili dalla difesa.

Le **metodologie standardizzate** — OSSTMM, OWASP, NIST 800-115, ISSAF, PCI DSS — garantiscono che il PT sia riproducibile, misurabile, e difendibile legalmente. OWASP si concentra sulle applicazioni web; NIST 800-115 è la guida tecnica del governo USA; PCI DSS riguarda i sistemi di pagamento.

La struttura di un PT segue fasi che nel mondo militare si chiamano **kill chain**: Reconnaissance → Weaponization → Delivery → Exploitation → Installation → Command & Control → Actions on Objectives. Le prime due fasi (Reconnaissance ed Enumeration) sono di S1; le successive entrano in S3–S11. Il framework **MITRE ATT&CK** cataloga le tecniche usate in ciascuna fase da gruppi APT reali — è la “biblioteca” che i team di difesa consultano per sapere cosa aspettarsi da un attore specifico.

Rischio, difesa e superfici d'attacco

Prima di sapere come attaccare, bisogna capire perché vale la pena difendersi. La formula è: **RI-SCHIO = PROBABILITÀ × IMPATTO**. Un sistema che espone una vulnerabilità critica su internet ha probabilità alta e impatto potenzialmente devastante; una vulnerabilità interna su un sistema isolato ha un profilo molto diverso. La **window of exposure** è il tempo che passa dalla scoperta di una vulnerabilità alla sua correzione — più è lunga, più è pericoloso. Gli attacchi **zero-day** sfruttano vulnerabilità senza patch: la window of exposure è teoricamente infinita per la vittima.

Il paesaggio delle minacce reali (dati ENISA 2023) vede in testa il ransomware (31.32%), gli attacchi DDoS (21.4%), e le violazioni di dati (20.09%). Tre casi concreti illustrano la complessità degli attacchi moderni:

- **Stuxnet**: malware che ha sabotato fisicamente le centrifughe di arricchimento dell'uranio iraniane modificando i PLC Siemens. Primo caso documentato in cui un attacco cyber ha prodotto effetti fisici nel mondo reale.
- **Solarwinds (2020)**: attacco supply chain. Gli attaccanti hanno compromesso il processo di build di Orion (piattaforma di monitoraggio IT), inserendo una DLL malevola distribuita come aggiornamento legittimo a migliaia di organizzazioni. La persistenza era invisibile perché arrivava firmata dal vendor.
- **xz/liblzma (CVE-2024-3094)**: due anni di social engineering su un maintainer open source stanco per inserire una backdoor in una libreria presente in quasi tutte le distro Linux. Dimostrazione che la supply chain open source è vulnerabile quanto quella commerciale.

La difesa si struttura secondo il **NIST Cybersecurity Framework (CSF)**: Identify → Protect → Detect → Respond → Recover. Non è un processo lineare ma ciclico. Le **politiche** dichiarano cosa è consentito o proibito (“nessun accesso remoto senza MFA”); i **meccanismi** sono gli strumenti tecnici che le fanno rispettare (firewall, IDS, autenticazione). Una politica senza meccanismo è carta; un meccanismo senza politica è rumore.

La **superficie d'attacco** ha tre vettori: fisico (accesso fisico ai locali e alle macchine), cyber (rete, applicazioni, sistemi), umano (phishing, social engineering). S1 si concentra sul vettore cyber; S6 affronta fisico e cloud; il social engineering si intreccia lungo tutto il programma.

Reconnaissance: l'attaccante conosce meglio di te la tua rete

La reconnaissance è la fase in cui l'attaccante raccoglie informazioni sull'obiettivo senza ancora interagire direttamente con i suoi sistemi. Si divide in **passiva** (fonti pubbliche, nessun contatto col target — lascia meno tracce) e **attiva** (interazione diretta: ping, port scan). In un PT si inizia sempre dalla passiva: spesso fornisce già abbastanza informazioni per pianificare l'attacco senza avvisare il target.

Google Dorking

Google è involontariamente un motore di ricerca di vulnerabilità. Tutto ciò che Google ha indicizzato è stato esposto — volontariamente o per errore — e un attaccante lo trova come chiunque altro. I **Google Dork** sono query costruite con operatori avanzati:

- `site:ulisse.unibo.it filetype:PDF intext:password` — trova PDF su un dominio specifico che contengono la parola “password”
- `site:ulisse.unibo.it intitle:index.of id_rsa -id_rsa.pub` — cerca directory listing con chiavi SSH private esposte (l’operatore `-` esclude i file `.pub`)

Gli operatori fondamentali: `site:` (limita a un dominio), `filetype:` (tipo di file), `intext:` (parola nel contenuto), `intitle:` (parola nel titolo), `inurl:` (parola nell’URL), `cache:` (versione in cache di Google), `link:` (pagine che linkano a un URL). Il database Google Hacking (exploit-db.com/google-hacking-database) cataloga dork già noti per trovare configurazioni errate comuni.

Dal lato difensore, la contromisura è il file `robots.txt`: dice ai crawler cosa non indicizzare. `User-agent: * / Disallow: /` blocca tutto; `Disallow: /cartella-sensibile` blocca cartelle specifiche. Ma `robots.txt` è una **convenzione, non un meccanismo di sicurezza** — un crawler ostile lo ignora, e la stessa lista delle cartelle escluse può rivelare all’attaccante cosa c’è di interessante. La vera difesa è non esporre mai informazioni sensibili su web server pubblici.

OSINT su IP e domini

Ogni host ha un indirizzo IP. I blocchi IP sono assegnati dallo **IANA** ai **RIR** (Regional Internet Registries — RIPE per l’Europa), che li sub-allocano ai **LIR** (Local Internet Registries: università, ISP). Conoscere anche solo un IP di un sistema dell’obiettivo permette di risalire via RIPE a tutti i blocchi allocati a quell’organizzazione — es. da `www.unibo.it` si raggiungono tutte le reti degli enti di ricerca italiani.

DNS enumeration: la mappa dell’infrastruttura

Il DNS è l’elenco telefonico di internet, ma anche una miniera di informazioni organizzative. I **record DNS** rivelano molto più degli semplici IP:

Tipo	Cosa rivela
A	hostname → IPv4
AAAA	hostname → IPv6
CNAME	alias (rivela nomi interni, CDN usati)
MX	mail server (rivela infrastruttura email)
NS	nameserver autoritativi (rivela provider DNS, cloud)
SOA	zona + email admin del dominio
TXT	SPF (quali server sono autorizzati a mandare email per il dominio) → mappa infrastruttura cloud
PTR	reverse lookup: da IP a hostname (rivela nomi interni di macchine)
SRV	service locator (es. SIP, XMPP — rivela servizi non-web)

La **DNS enumeration** è il processo di trovare tutti questi record per un target. Il tool di base è `nslookup`: `nslookup google.com` chiede il record A; `nslookup -type=any google.com` chiede tutti i tipi disponibili. Per approcci più sistematici: `dnsrecon -d DOMAIN` e `dnsmap DOMAIN` includono guessing e forza bruta sui sottodomini.

Un **aggravante pericoloso**: se un nameserver risponde a richieste di **zone transfer (AXFR)**, restituisce l'intera zona DNS in una query — tutta la struttura interna dell'organizzazione in un colpo solo. Un altro aggravante: i record DNS rimossi possono sopravvivere nelle cache dei resolver per tutta la durata del TTL — un sottodominio cancellato può restare raggiungibile per ore o giorni.

Subdomain enumeration

I sottodomini sono una delle superfici d'attacco più trascurate. Dietro a `legacy.api.example.com` o `old-staging.example.com` può nascondersi un'applicazione dimenticata, non aggiornata, esposta su una porta non-standard.

Distinzione critica: `http://myaltropc.ulisse.com` **non è un subdomain DNS** — è un'applicazione web ospitata sulle porte 80/443 del rispettivo host. Un subdomain DNS come `corsosec.ulisse.com` invece potrebbe non avere nulla su 80/443 ma esporre un servizio completamente diverso su porta 8080 o altra. È quindi necessario: (1) mappare tutti i subdomain DNS, (2) per ciascuno analizzare separatamente quali servizi espone e su quali porte.

La strategia **passiva** interroga dataset pubblici già esistenti — SecurityTrails, Shodan, Censys, VirusTotal, Binaryedge — senza mai toccare il target. Tool come **amass** (OWASP), **subfinder**, **assetfinder** e **Findomain** wrappano queste fonti in un unico output.

Un vettore passivo sottovalutato è il **Certificate Transparency (CT) abuse**: ogni certificato TLS emesso da una CA viene registrato in log pubblici e immutabili (RFC 6962). Interrogando `crt.sh/?q=%25.example.com` si ottengono tutti i sottodomini che hanno mai avuto un certificato TLS — inclusi quelli rimossi dal DNS ma che potrebbero ancora essere raggiungibili. Non esiste difesa contro il CT abuse: la trasparenza è per design.

La strategia **attiva** include DNS bruteforcing, permutation/alternations, VHOST probing, e recursive enumeration. Non esiste una vera difesa dalla subdomain enumeration (i subdomain sono per definizione pubblici), ma conoscere la propria superficie esposta è il primo passo difensivo.

Shodan.io è il motore di ricerca per dispositivi connessi. Mentre Google indicizza contenuto HTML, Shodan indicizza i banner dei servizi esposti su internet: porte aperte, versione del software, geolocalizzazione, organizzazione. Esempio reale dalle slide: IP 137.204.24.147, UNI-Bologna, porta 80 TCP, nginx (HTTP 1.1 301 Moved Permanently).

nmap: vedere cosa espone davvero un host

nmap è lo strumento centrale della fase di enumerazione attiva. Dopo aver individuato blocchi IP e domini via OSINT, si verifica quali host sono effettivamente raggiungibili e quali servizi espongono.

La prima operazione è il **host discovery**: `sudo nmap -sn 192.168.X.0/24` scopre gli host attivi nella subnet. Con `sudo` usa ARP su rete locale — più affidabile e veloce; senza privilegi, cade su ICMP e TCP (più lento, spesso bloccato dai firewall).

Errore frequente: `nmap -sn` senza `sudo` su reti host-only — senza privilegi non può usare ARP, i risultati possono essere incompleti o assenti.

Scoperto un host, si cercano le **porte aperte**. `nmap <IP>` senza opzioni scansiona le ~1000 porte “più popolari”. Ma molti servizi usano porte non-standard — un servizio sulla porta 1337 non appare in questo scan.

Errore frequente: dimenticarsi `-p-` significa perdere servizi su porte alte. Quando si vuole la mappa completa di un host, serve `nmap -p- <IP>`.

Per sapere non solo che una porta è aperta ma anche quale servizio specifico è in ascolto e la sua versione, si usa `-sV`:

Errore frequente: `-sT` e `-sV` non sono equivalenti. `-sT` è il TCP connect scan: dice “porta aperta o chiusa”, niente di più. `-sV` completa la connessione e invia probe applicativi per leggere il **banner** del servizio, identificando nome e versione. Per l’enumeration vera serve `-sV`.

Errore frequente: le porte nella lista devono essere separate da virgola: `-p 22,80,3306`. Spazi tra i numeri fanno interpretare i numeri extra come host aggiuntivi — `nmap -p 22 80 3306 192.168.1.1` scansiona tre host diversi, non tre porte.

Dietro a `nmap` c’è un meccanismo che vale la pena capire: in modalità SYN scan (default con `sudo`), invia un SYN e aspetta un SYN-ACK (porta aperta) o RST (porta chiusa) senza mai completare il three-way handshake. Questo lo rende meno rumoroso e meno loggato rispetto al TCP connect scan (`-sT`). La versione detection (`-sV`) completa invece la connessione perché deve inviare payload applicativi per leggere la risposta — il banner poi si mappa su CVE noti per trovare vulnerabilità specifiche della versione.

unicornscan è un’alternativa con fingerprinting più affidabile, più veloce (separa fase di invio e ricezione), e può salvare le risposte per analisi successive con altri strumenti. In scenari dove la velocità è critica o dove si vuole evitare il reverse fingerprinting sull’OS d’origine, `unicornscan` è preferibile.

Evasione: scan senza essere visti

La fase di enumeration genera traffico visibile: IDS e IPS possono rilevare scansioni aggressive e bloccarle o loggare l’IP sorgente. L’evasione serve a tre scopi: testare l’efficacia delle difese (IDS), evitare blocchi che comprometterebbero il test (IPS), e condurre reconnaissance senza lasciare tracce.

La **reconnaissance anonima** usa ToR, account usa-e-getta sui siti interrogati, e VM periodicamente sostituite per non esporre l’IP reale. La **scansione stealth** configura il timing di `nmap` (`-T0` ultra-lento → `-T5` aggressivo), randomizza l’ordine degli indirizzi e delle porte, e — con `unicornscan` — evita di usare lo stack TCP/IP dell’host d’origine per prevenire il **reverse fingerprinting**: il target non riesce a identificare l’OS dell’attaccante dal modo in cui costruisce i pacchetti.

L’**enumerazione adattativa** rispetto a firewall, IDS e IPS è una disciplina a sé: se un firewall blocca certi tipi di pacchetti, si prova con TCP fragment, decoy scan, source port spoofing. `nmap` ha flag specifici per tutto questo (riferimento: nmap.org/book/firewalls.html).

Tentativi di accesso ai servizi

Dopo aver enumerato host e servizi, si analizzano i **protocolli applicativi** più comuni: SMB, SMTP, SNMP. Ogni protocollo può esporre informazioni aggiuntive o permettere accesso a dati. Il **fuzzing applicativo** invia payload randomizzati (strumenti: `bed`, `doona`) per sollecitare risposte

impreviste — crash, info leak, comportamenti anomali. I **framework per exploit** (Metasploit e simili) automatizzano lo sviluppo e l'esecuzione delle fasi successive.

Vulnerability scanners: da “porta aperta” a “vulnerabilità sfruttabile”

Identificati host e servizi, si passa a cercarne le vulnerabilità. Gli scanner completi — **Nessus** (Tenable, commerciale) e **OpenVAS / Greenbone Community Edition (GCE)** (open-source) — automatizzano questa fase: per ogni servizio trovato, eseguono test tramite plug-in caricabili e collegano ogni finding ai **CVE** corrispondenti.

OpenVAS nasce come fork open-source di Nessus nel 2005 (quando Tenable passò a licenza proprietaria). Dal 2008 è sviluppato attivamente da Greenbone, ora chiamato **Greenbone Community Edition (GCE)**. L'architettura: OpenVAS Scanner e Notus Scanner mandano probe verso i target → i risultati fluiscono tramite `osspd-openvas` al `gvmd` (Greenbone Vulnerability Management Daemon) → l'UI è il **Greenbone Security Assistant** (interfaccia web), accessibile anche via GMP (API). Il feed — aggiornato ogni giorno — contiene **Network Vulnerability Tests (NVT)**: per ogni vulnerabilità, descrizione, piattaforme colpite, e processo di verifica.

Postura interna vs esterna

Reconnaissance ed enumeration si fanno tipicamente dall'esterno — postura fedele all'attaccante. Ma dall'esterno ci sono NIDS e firewall di perimetro che possono bloccare o rallentare. Una variante potente è l'**auto-attacco dall'interno**: si simula un attaccante già infiltrato. NIDS e FW perimetrali vengono scavalcati, i sistemi interni sono raggiungibili senza ostacoli, e i test per gli **HIDS** (Host-based IDS, S11) sono molto più efficaci. In S1 si parte dall'esterno; S7–S11 esplorano la postura interna, incluse iniezione di software e occultamento di processi.

Connessioni

- **Con SysAdmin 3D**: `ss -tlnp` in 3D mostra le stesse porte che nmap vedrà dall'esterno — la prospettiva si ribalta. Una porta in ascolto su 0.0.0.0 in SysAdmin è una porta che nmap trova in S1.
 - **Con S2 (Autenticazione)**: `nmap -sV` rivela i servizi; S2 mostra come attaccarli a livello di credenziali (brute force, hash cracking).
 - **Con S3 (Web Security)**: la subdomain enumeration porta a web app dimenticate su porte non-standard — quelle stesse app vengono attaccate in S3 con OWASP Top 10.
 - **Con S5 (Firewall)**: la scansione stealth di nmap è il punto di vista opposto rispetto alle regole iptables di S5 — capire come il firewall filtra le porte aiuta a capire perché certi scan vengono bloccati.
 - **Con S10 (Suricata)**: ogni scan nmap genera una firma nel traffico riconoscibile. In S1 si genera quel traffico; in S10 si impara a catturarlo e a scrivere regole per rilevarlo.
-

Domande di autoverifica (stile quiz teorico)

All'esame c'è penalità per risposta sbagliata. Se non sei sicuro, lascia in bianco.

1. Un penetration test differisce da una vulnerability assessment perché: - a) usa solo strumenti automatici - b) è sempre condotto da team esterni all'organizzazione - c) verifica effettivamente la sfruttabilità delle vulnerabilità trovate - d) non richiede autorizzazione scritta
2. La query Google `site:example.com filetype:PDF intext:password` trova: - a) qualsiasi file su example.com - b) PDF su example.com che contengono la parola "password" nel testo - c) pagine di example.com che parlano di PDF e password - d) PDF indicizzati da qualsiasi sito che menzionano example.com
3. Il record DNS di tipo MX rivela: - a) l'indirizzo IPv6 dell'host - b) i mail server dell'organizzazione - c) il nameserver autoritativo della zona - d) un alias per un altro hostname
4. Vero o Falso: `nmap -sT` e `nmap -sV` forniscono lo stesso livello di informazione sui servizi in ascolto.
5. La Certificate Transparency (CT) abuse permette di: - a) decifrare il traffico HTTPS intercettato - b) enumerare i sottodomini di un dominio cercando nei log CT pubblici - c) falsificare certificati TLS per qualsiasi dominio - d) bloccare il rinnovo dei certificati di un competitor
6. Vero o Falso: `robots.txt` con `Disallow: /` impedisce agli attaccanti di accedere alle cartelle escluse.

Risposte: 1-c · 2-b · 3-b · 4-Falso (-sT = porta aperta/chiusa; -sV = porta + servizio + versione via banner) · 5-b · 6-Falso (`robots.txt` è una convenzione per crawler legittimi; un attaccante lo ignora)

Collegati

Hub: [\[\[master_map_studio\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[metodo_studio_esami_pratici\]\]](#)